Arquitectura de Computadoras

# Trabajo Práctico N° 2

Comunicación UART


**Integrantes**

De la Mata, Nicolás

Quispe, Mateo


**Repositorio**

https://github.com/Tuteku/TP-Arquitectura-comp

# Índice

**1. Objetivo** **2**

**2. Descripción general** **2**

2.1. Diagrama de bloques . . . . . 2

2.2. Formato de la trama y protocolo . . . . . 3

**3. Generador de baudios** **3**

**4. Receptor (`uart_rx`)** **3**

**5. Transmisor (`uart_tx`)** **4**

**6. Circuito de interfaz (`uart_interface`)** **6**

**7. Bloque ALU (`top_uart_alu`)** **7**

**8. Módulo `top_level`** **9**

**9. Decisiones de diseño** **10**

**10. Interfaz con la placa** **10**

10.1. Prueba sobre la placa . . . . . 11

**11. Verificación** **11**

11.1. Receptor . . . . . 11

11.2. Transmisor . . . . . 12

11.3. Interfaz . . . . . 12

11.4. Sistema completo . . . . . 13

11.5. Resultados . . . . . 14

**12. Análisis de tiempo: por qué alcanza con buffer + flag** **15**

**13. Conclusiones** **17**

# 1. Objetivo

Implementar un módulo UART sobre FPGA y usarlo para comunicar la ALU del trabajo anterior con una PC. La PC envía tres bytes por el puerto serie (operando A, operando B y opcode) y la placa responde con un byte con el resultado. El diseño debía ser parametrizable y cada módulo debía validarse con su propio test bench.

Se utilizó una **Digilent Basys3** (Artix-7 `xc7a35tcpg236-1`) y **Vivado 2025.2**.

# 2. Descripción general

El sistema se dividió en siete módulos, siguiendo el esquema de la consigna:

- **`baud_rate_generator`**: genera un pulso (`tick`) a 16 veces la velocidad de transmisión.
- **`uart_rx`**: receptor. Detecta el bit de start, sobremuestrea cada bit y arma el byte.
- **`uart_tx`**: transmisor. Serializa un byte agregando los bits de start y stop.
- **`uart_interface`**: circuito de interfaz con **buffer + flags**. Guarda el último byte recibido y el próximo byte a transmitir, e indica su estado con las banderas `rx_empty` y `tx_full`.
- **`top_uart_alu`**: bloque ALU del esquema. Una máquina de estados lee A, B y el opcode desde la interfaz, los guarda en registros y escribe el resultado de vuelta.
- **`alu`**: la ALU del trabajo anterior, sin modificaciones, instanciada dentro de `top_uart_alu`.
- **`top_level`**: instancia y conecta los módulos anteriores.

## 2.1. Diagrama de bloques

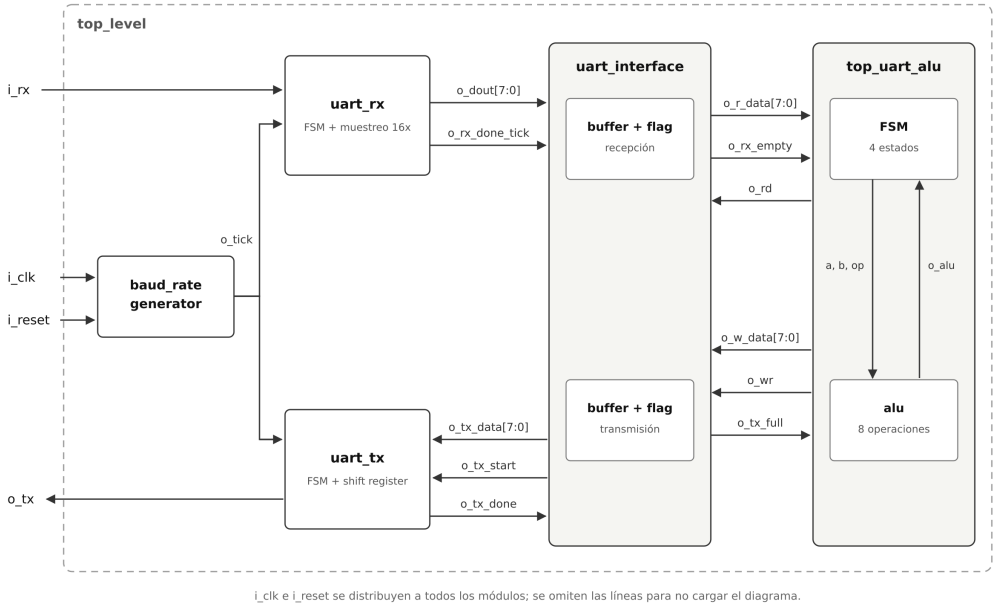



**Figura 1:** Diagrama de bloques del sistema.

### 2.2. Formato de la trama y protocolo

Se usó el formato **8N1** (un bit de start en `0`, ocho bits de datos empezando por el LSB, sin paridad y un bit de stop en `1`) a **19200 baudios**, es decir unos 52 µs por bit.

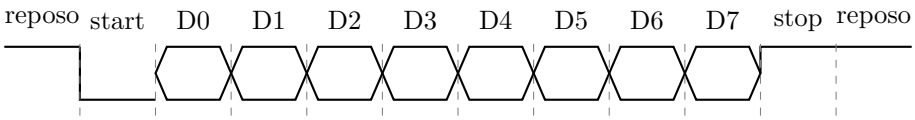



**Figura 2:** Trama 8N1: el LSB (D0) se transmite primero.

| Orden | Sentido | Contenido |
|---|---|---|
| 1 | PC → FPGA | Operando A |
| 2 | PC → FPGA | Operando B |
| 3 | PC → FPGA | Opcode (6 bits menos significativos) |
| 4 | FPGA → PC | Resultado de la ALU |

**Tabla 1:** Protocolo de una operación.

## 3. Generador de baudios

```
module baud_rate_generator #(
        parameter CLK_FREQ  = 100000000,
        parameter BAUD_RATE = 19200
    )
    (
        input  wire i_clk,
        input  wire i_reset,
        output wire o_tick
    );
```

Es un módulo contador `M` cuya salida `o_tick` vale `1` durante un ciclo cada vez que llega a `M-1`:

```
localparam M        = (CLK_FREQ/(BAUD_RATE*16));
localparam NB_COUNT = $clog2(M);
```

Con 100 MHz y 19200 baudios resulta `M = 325` (325,52 truncado), lo que da una velocidad real de 19 230,8 baudios, un error de +0,16 %. Como el receptor muestrea en el centro de cada bit, tolera hasta medio bit de desfase acumulado en la trama, así que este error no afecta la comunicación.

## 4. Receptor (`uart_rx`)

```
module uart_rx #(
    parameter NB_DATA = 8,
    parameter SB_TICK = 16
)
(
    input  wire                 i_s_tick,
```

```
    input  wire                 i_clk,
    input  wire                 i_reset,
    input  wire                 i_rx,
    output wire [NB_DATA-1:0]   o_dout,
    output wire                 o_rx_done_tick
);
```

El receptor usa 16 ticks por bit. Al detectar el flanco de bajada del start cuenta 8 ticks para ubicarse en el **centro del bit de start**, y desde ahí cuenta de a 16, de modo que cada muestra cae en el centro del bit siguiente.

| Estado | Acción | Transición |
|---|---|---|
| `IDLE` | Espera que la línea baje a `0` | A `START` cuando `i_rx = 0` |
| `START` | Cuenta 8 ticks | A `DATA` cuando `s_reg = 7` |
| `DATA` | Cada 16 ticks desplaza `i_rx` dentro de `b_reg` | A `STOP` después del bit `NB_DATA-1` |
| `STOP` | Espera 16 ticks | A `IDLE`, generando `o_rx_done_tick` |

**Tabla 2:** Estados del receptor.

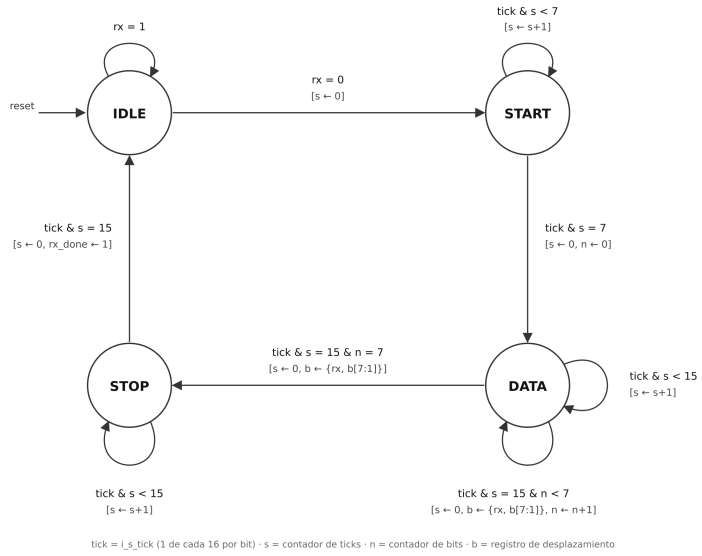



**Figura 3:** Máquina de estados de `uart_rx`.

`s_reg` cuenta ticks, `n_reg` cuenta bits y `b_reg` es el registro de desplazamiento. Como llega primero el LSB, cada bit entra por la izquierda y el registro se desplaza a la derecha:

```
b_next = {i_rx, b_reg[NB_DATA-1:1]};
```

## 5. Transmisor (`uart_tx`)

```
module uart_tx #(
    parameter NB_DATA = 8,
```

```
    parameter SB_TICK = 16
)
(
    input  wire [NB_DATA-1:0] i_data,
    input  wire               i_tx_start,
    input  wire               i_clk,
    input  wire               i_s_tick,
    input  wire               i_reset,
    output wire               o_tx,
    output wire               o_tx_done
);
```

| Estado | Línea | Transición |
|---|---|---|
| `IDLE` | `1` | A `START` con `i_tx_start`, cargando `i_data` en `b_reg` |
| `START` | `0` | A `DATA` después de 16 ticks |
| `DATA` | `b_reg[0]` | Cada 16 ticks desplaza `b_reg`; a `STOP` después del bit `NB_DATA-1` |
| `STOP` | `1` | A `IDLE` después de 16 ticks, generando `o_tx_done` |

**Tabla 3:** Estados del transmisor.

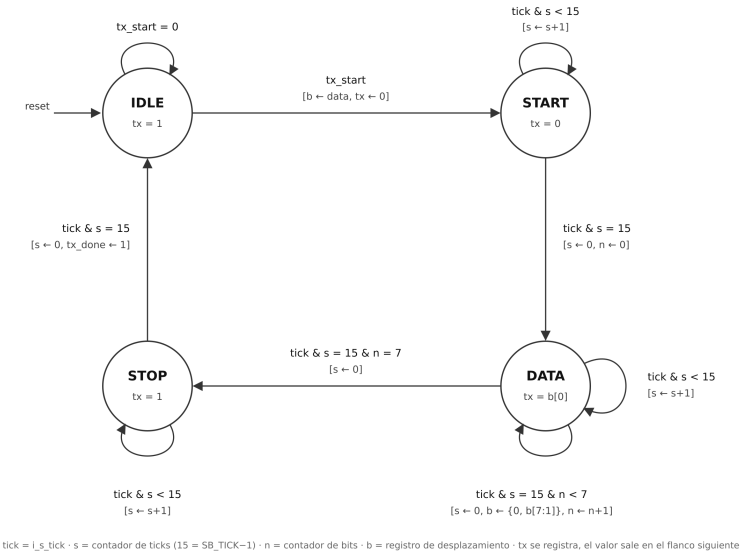



**Figura 4:** Máquina de estados de `uart_tx`.

El byte se copia en `b_reg` al recibir `i_tx_start`, así que `i_data` puede cambiar durante la transmisión. Siempre se transmite el bit 0 del registro, que se desplaza a la derecha después de cada bit:

```
o_tx_next = b_reg[0];
...
b_next = {1'b0, b_reg[NB_DATA-1:1]};
```

## 6. Circuito de interfaz (`uart_interface`)

```
module uart_interface #(
    parameter NB_DATA = 8
)
(
    input  wire               i_clk,
    input  wire               i_reset,
    // lado UART
    input  wire [NB_DATA-1:0] i_rx_data,   // d_out del Rx
    input  wire               i_rx_done,
    input  wire               i_tx_done,
    output wire [NB_DATA-1:0] o_tx_data,   // d_in del Tx
    output wire               o_tx_start,
    // lado ALU
    output wire [NB_DATA-1:0] o_r_data,
    input  wire               i_rd,
    output wire               o_rx_empty,
    input  wire [NB_DATA-1:0] i_w_data,
    input  wire               i_wr,
    output wire               o_tx_full
);
```

El receptor y el transmisor avisan con pulsos de un ciclo (`rx_done`, `tx_done`). Si el módulo que los consume no está en el estado justo en ese ciclo, el evento se pierde. La interfaz desacopla ambos lados con dos buffers de una palabra, cada uno con un flip-flop de bandera que convierte el pulso en un nivel que se mantiene hasta que el dato se consume:

| Buffer | Se llena con | Se vacía con | Bandera hacia la ALU |
|---|---|---|---|
| RX (`rx_buf`, `rx_flag`) | `i_rx_done`, guardando `i_rx_data` | `i_rd` | `o_rx_empty = ~rx_flag` |
| TX (`tx_buf`, `tx_flag`) | `i_wr`, guardando `i_w_data`, solo si el buffer está libre | `i_tx_done` | `o_tx_full = tx_flag` |

**Tabla 4:** Buffers de la interfaz.

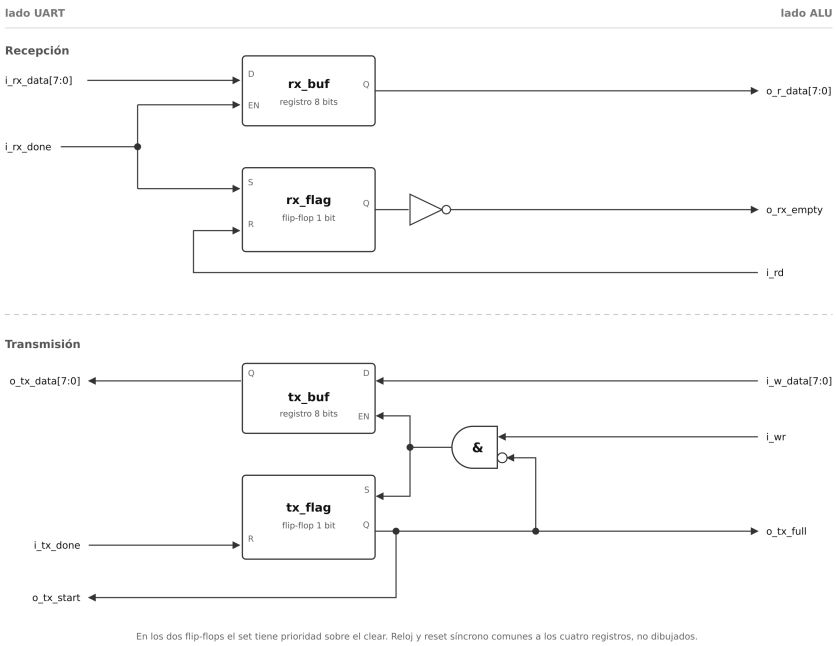



**Figura 5:** Estructura interna de `uart_interface`.

```
// Buffer RX: se llena con rx_done, se vacía con rd
if (i_rx_done) begin
    rx_buf  <= i_rx_data;
    rx_flag <= 1'b1;
end
else if (i_rd) begin
    rx_flag <= 1'b0;
end
// Buffer TX: se llena con wr, se vacía con tx_done
if (i_wr && !tx_flag) begin
    tx_buf  <= i_w_data;
    tx_flag <= 1'b1;
end
else if (i_tx_done) begin
    tx_flag <= 1'b0;
end
```

- Si llegan `i_rx_done` e `i_rd` en el mismo ciclo, tiene prioridad la escritura, para no perder el byte nuevo.
- Un `i_wr` con el buffer de TX lleno se ignora, así no se pisa un byte que todavía no se transmitió.
- `o_tx_start` es directamente `tx_flag`. El `uart_tx` solo mira `i_tx_start` en `IDLE`, y la bandera baja con `tx_done` en el mismo flanco en que el transmisor vuelve a `IDLE`, así que cada byte se transmite una sola vez.

# 7. Bloque ALU (`top_uart_alu`)

```
module top_uart_alu #(
    parameter NB_DATA = 8,
    parameter NB_OP   = 6
)
(
    input  wire               i_clk,
    input  wire               i_reset,
    input  wire [NB_DATA-1:0] i_r_data,
    input  wire               i_rx_empty,
    input  wire               i_tx_full,
    output wire               o_rd,
    output wire [NB_DATA-1:0] o_w_data,
    output wire               o_wr
);
```

Reemplaza a los registros con enable del `alu_top` del trabajo anterior. En lugar de los botones, una máquina de estados toma cada byte disponible en la interfaz y lo carga en el registro que corresponde según el orden de llegada. La salida de la `alu` instanciada adentro es `o_w_data`.

| Estado | Acción | Transición |
|---|---|---|
| `RX_A` | Guarda `i_r_data` en `reg_a` y genera `rd` | A `RX_B` cuando `i_rx_empty = 0` |
| `RX_B` | Guarda `i_r_data` en `reg_b` y genera `rd` | A `RX_OP` cuando `i_rx_empty = 0` |
| `RX_OP` | Guarda los 6 bits bajos de `i_r_data` en `reg_op` y genera `rd` | A `TX_SEND` cuando `i_rx_empty = 0` |
| `TX_SEND` | Genera `wr` con el resultado de la ALU en `o_w_data` | A `RX_A` cuando `i_tx_full = 0` |

**Tabla 5:** Estados de `top_uart_alu`.

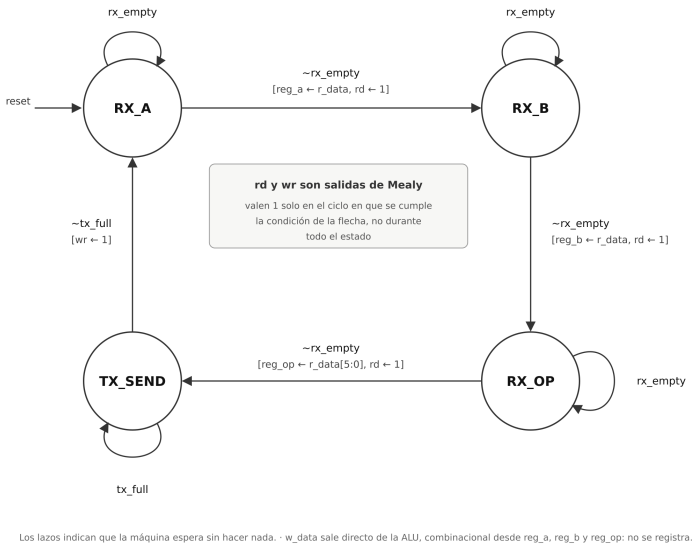



**Figura 6:** Máquina de estados de `top_uart_alu`.

El estado `TX_SEND` existe para darle un ciclo a la ALU: al salir de `RX_OP` el opcode recién se escribe en `reg_op`, por lo que el resultado correcto aparece en el ciclo siguiente. Sin este estado

se escribiría el resultado con el opcode anterior.

Una vez que el resultado queda en el buffer de TX, la máquina vuelve a `RX_A` y puede recibir la siguiente operación mientras el transmisor todavía está enviando.

`rd` y `wr` son **combinacionales**: se asignan en el `always @(*)` y salen directo por `o_rd` y `o_wr`. Si `rd` estuviera registrado, la bandera de RX bajaría un ciclo tarde, la máquina vería `i_rx_empty = 0` también en `RX_B` y cargaría el mismo byte como A y como B.

```
assign o_rd = rd;
assign o_wr = wr;
```

## 8. Módulo `top_level`

Solo conecta los módulos según el esquema. Un único generador de baudios alimenta al receptor y al transmisor. El receptor y el transmisor se conectan a la interfaz (`d_out`, `rx_done`, `d_in`, `tx_start`, `tx_done`), y la interfaz se conecta con el bloque ALU (`r_data`, `rd`, `rx_empty`, `w_data`, `wr`, `tx_full`).

```
module top_level #(
    parameter CLK_FREQ  = 100000000,
    parameter BAUD_RATE = 19200,
    parameter NB_DATA   = 8,
    parameter SB_TICK   = 16,
    parameter NB_OP     = 6
)
(
    input  wire i_rx,
    input  wire i_clk,
    input  wire i_reset,
    output wire o_tx
);
```

Todos los parámetros se propagan a los submódulos, así que la velocidad o el ancho de datos se cambian desde un único lugar.

Los puertos del `top_level` no cambiaron respecto a la versión sin buffer, así que el archivo de restricciones y el test bench del sistema completo siguen sirviendo.

El esquemático RTL que genera Vivado confirma que las conexiones coinciden con el diagrama de bloques de la Figura 1:

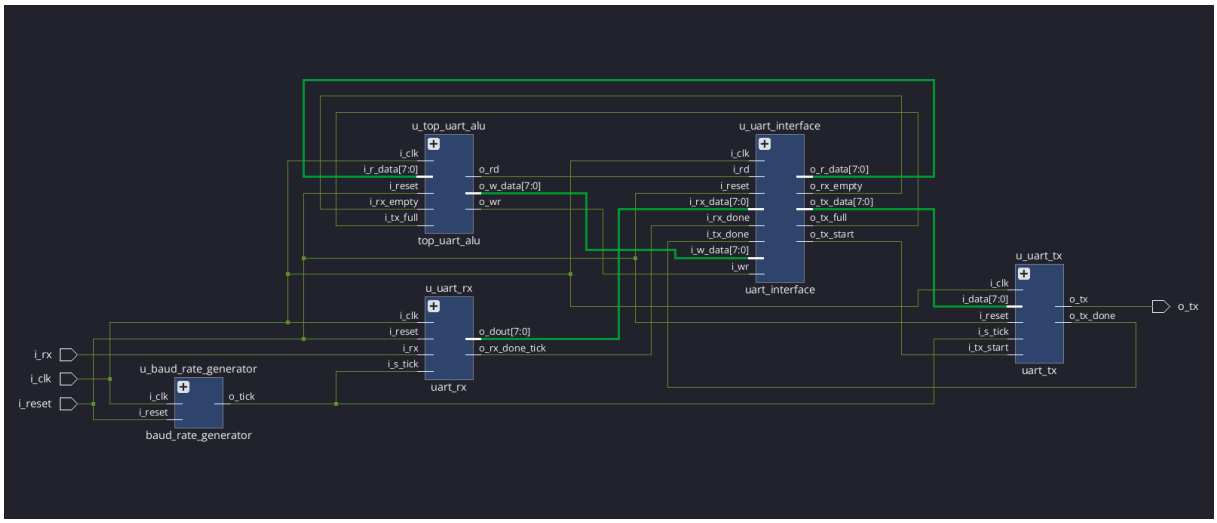



**Figura 7:** Esquemático RTL de `top_level` generado por Vivado.

## 9. Decisiones de diseño

- **FSM con dos bloques `always`**: un bloque secuencial con los registros y el reset, y uno combinacional (`always @(*)`) que calcula el estado siguiente. Al inicio del bloque combinacional se asignan valores por defecto a todas las señales `_next`, lo que evita la inferencia de latches.
- **El tick como habilitación, no como reloj**: todo el diseño trabaja con el reloj de 100 MHz y el tick se consulta con `if (i_s_tick)`. Así hay un único dominio de reloj y el análisis de tiempo cubre todo el diseño.
- **Un único generador de baudios** compartido por receptor y transmisor.
- **Salida del transmisor registrada**: `o_tx` sale de un flip-flop (`o_tx_reg`), lo que evita glitches que la PC podría interpretar como un bit de start. El reset la deja en `1`.
- **Pulsos de un ciclo convertidos en banderas**: `o_rx_done_tick` y `o_tx_done` duran un ciclo. La interfaz los convierte en las banderas `rx_empty` y `tx_full`, que se mantienen hasta que el dato se consume, así que el bloque ALU no necesita estar esperando en el ciclo exacto del pulso.
- **Lectura y escritura combinacionales**: `rd` y `wr` se generan en el mismo ciclo en que la máquina de estados ve la bandera, de modo que cada byte se lee una sola vez.
- **Separación UART / control**: la interfaz no conoce el protocolo A → B → Op. Solo almacena bytes, y el orden lo resuelve `top_uart_alu`. Así se puede cambiar el protocolo sin tocar la UART ni la interfaz.
- **Reset síncrono** activo en alto.

## 10. Interfaz con la placa

La Basys3 incluye un conversor USB-UART (FTDI FT2232), así que la comunicación usa el mismo cable USB de programación.

| Puerto | Pin | Elemento físico |
|---|---|---|
| `i_rx` | B18 | RsRx (PC → FPGA) |
| `o_tx` | A18 | RsTx (FPGA → PC) |
| `i_reset` | T18 | BTNU |
| `i_clk` | W5 | Oscilador de 100 MHz |

**Tabla 6:** Asignación de pines.

### 10.1. Prueba sobre la placa

Para usar el sistema desde la PC se escribió un script en Python (`uart_sim/alu_term.py`, con la librería `pyserial`). Abre el puerto serie a 19200 baudios 8N1, pide la operación por teclado (por ejemplo `ADD 5 3`), manda los tres bytes y espera el byte de respuesta.

```
python3 alu_term.py /dev/ttyUSB1          # modo interactivo
```

## 11. Verificación

| Test bench | Módulo | Qué verifica |
|---|---|---|
| `tb_uart_rx` | `uart_rx` | Recepción y armado de un byte |
| `tb_uart_tx` | `uart_tx` | Serialización de un byte en la línea |
| `tb_interface` | `uart_interface` | Llenado y vaciado de los buffers y sus banderas |
| `tb_top_level` | `top_level` | Una operación completa por la línea serie |

**Tabla 7:** Bancos de prueba.

En `tb_uart_rx` y `tb_uart_tx` el tick se genera dentro del propio banco cada 8 ciclos de reloj (en lugar de 325) para acortar la simulación. Como los módulos solo cuentan ticks, el comportamiento es el mismo:

```
integer cnt = 0;
always @(posedge clk) begin
    if (cnt == 7) begin
        cnt    <= 0;
        s_tick <= 1;
    end
    else begin
        cnt    <= cnt + 1;
        s_tick <= 0;
    end
end
```

### 11.1. Receptor

La `task send_bit` mantiene un valor en la línea durante 16 ticks:

```
task send_bit (input valor);
    integer k;
```

```
    begin
        @(negedge clk);
        rx = valor;
        k = 0;
        while (k < 16) begin
            @(posedge clk);
            if (s_tick) k = k + 1;
        end
    end
endtask
```

Con ella se envía la trama de `0x4B` (`01001011`) y luego se muestra `o_dout`:

```
data = 8'b01001011;
...
send_bit(0);                                // start
for (i = 0; i < 8; i = i + 1)
    send_bit(data[i]);                      // datos, D0 primero
send_bit(1);                                // stop

#200;
$display("Enviado 0x4B, recibido 0x%02h", dout);
```

Se eligió `0x4B` porque no es simétrico: si el receptor invirtiera el orden de los bits se obtendría `0xD2`.

## 11.2. Transmisor

Se da un pulso de un ciclo en `i_tx_start` con el dato `0x4B`, se espera el bit de start, se avanzan 8 ticks hasta su centro y se toma una muestra cada 16 ticks, igual que el `uart_rx`:

```
@(negedge clk) tx_start = 1;
@(negedge clk) tx_start = 0;

wait (tx == 1'b0);
wait_ticks(8);
for (i = 0; i < 8; i = i + 1) begin
    wait_ticks(16);
    recibido[i] = tx;       // D0 sale primero
end

wait (tx_done == 1'b1);
$display("Enviado 0x%02h, en la linea 0x%02h", dato, recibido);
```

## 11.3. Interfaz

El receptor, el transmisor y el bloque ALU se reemplazan por estímulos directos sobre los puertos de la interfaz. Cada evento es un pulso de un ciclo que cambia en el flanco de bajada, y el estado de los buffers se muestra en el ciclo siguiente, cuando ya quedó registrado:

```
// RX: llega 0x4B del Rx
@(negedge clk) begin rx_data = 8'h4B; rx_done = 1; end
@(negedge clk) rx_done = 0;
```

```
$display("RX lleno: r_data=0x%02h rx_empty=%b (esperado 0x4B, 0)", r_data, rx_empty)
    ;

// la ALU lee el dato
@(negedge clk) rd = 1;
@(negedge clk) rd = 0;
$display("RX leido: rx_empty=%b (esperado 1)", rx_empty);

// TX: la ALU escribe 0x08
@(negedge clk) begin w_data = 8'h08; wr = 1; end
@(negedge clk) wr = 0;
$display("TX lleno: tx_data=0x%02h tx_full=%b tx_start=%b (esperado 0x08, 1, 1)",
    tx_data, tx_full, tx_start);

// el Tx termina de transmitir
@(negedge clk) tx_done = 1;
@(negedge clk) tx_done = 0;
$display("TX libre: tx_full=%b tx_start=%b (esperado 0, 0)", tx_full, tx_start);
```

Se recorre el ciclo completo de cada buffer. En RX, `rx_done` llena el buffer y baja `rx_empty`, y `rd` lo vacía. En TX, `wr` llena el buffer y levanta `tx_full` y `tx_start`, y `tx_done` lo libera.

## 11.4. Sistema completo

El `top_level` se prueba solo a través de `i_rx` y `o_tx`, a 2 Mbaudios para acortar la simulación. La duración de un bit se calcula igual que en el generador de baudios, para tener en cuenta el truncamiento de la división:

```
localparam M      = CLK_FREQ / (BAUD_SIM * 16);   // 3
localparam BIT_NS = M * 16 * 10;                  // 480 ns
```

Las `task` `send_byte` y `recv_byte` arman y leen las tramas con retardos de `BIT_NS`. `recv_byte` espera el flanco de bajada del start y avanza un bit y medio para muestrear en el centro de `D0`:

```
task send_byte (input [7:0] dato);
    integer i;
    begin
        rx = 1'b0;                      // start
        #BIT_NS;
        for (i = 0; i < 8; i = i + 1) begin
            rx = dato[i];               // D0 primero
            #BIT_NS;
        end
        rx = 1'b1;                      // stop
        #BIT_NS;
    end
endtask

task recv_byte (output [7:0] dato);
    integer i;
    begin
        @(negedge tx);                  // start bit
        #(BIT_NS + BIT_NS/2);           // caer en el medio de D0
        for (i = 0; i < 8; i = i + 1) begin
            dato[i] = tx;
            #BIT_NS;
```

```
        end
    end
endtask
```

El envío y la recepción corren en paralelo usando dos bloques `initial`. En Verilog todos los `initial` arrancan en el instante cero y se ejecutan concurrentemente, por lo que no hace falta un `fork ... join`:

```
// escucha la respuesta desde el instante cero, en paralelo
initial begin
    recv_byte(resultado);
    $display("A=0x05  B=0x03  ->  respuesta=0x%02h (esperado 0x08)", resultado);
    #(BIT_NS*2);
    $finish;
end

// manda los tres bytes
initial begin
    rx    = 1'b1;
    reset = 1'b1;
    #100;
    reset = 1'b0;
    #100;

    send_byte(8'h05);
    #(BIT_NS*2);
    send_byte(8'h03);
    #(BIT_NS*2);
    send_byte({2'b00, 6'b100000});

    // timeout: si la respuesta no llega, corta igual
    #(BIT_NS*30);
    $display("TIMEOUT: no llego respuesta por tx");
    $finish;
end
```

El `$finish` lo ejecuta el bloque receptor apenas termina de leer la respuesta, así que la simulación no depende de estimar cuánto tarda la trama de vuelta. El bloque de envío ahora solo actúa como timeout, por si la respuesta no llega nunca.

## 11.5. Resultados

`tb_uart_rx`: el byte recibido coincide con el enviado.




**Figura 8:** Salida de consola de `tb_uart_rx`.

`tb_uart_tx`: el byte reconstruido desde la línea coincide con el dato de entrada.




**Figura 9:** Salida de consola de `tb_uart_tx`.

`tb_interface`: las banderas `rx_empty`, `tx_full` y `tx_start` y los datos `r_data` y `tx_data` toman los valores esperados en cada paso.

```
RX lleno: r_data=0x4b rx_empty=0 (esperado 0x4B, 0)
RX leido: rx_empty=1 (esperado 1)
TX lleno: tx_data=0x08 tx_full=1 tx_start=1 (esperado 0x08, 1, 1)
TX libre: tx_full=0 tx_start=0 (esperado 0, 0)
$finish called at time : 220 ns : File "/home/nicolas/Documents/Facultad/arqui/TP-Arquitectura-comp/ALU.srcs/sim_1/new/tb_interface.v" Line 90
```

**Figura 10:** Salida de consola de `tb_interface`.

`tb_top_level`: la respuesta en `o_tx` es `0x08`, el resultado de $5 + 3$.

```
run all
A=0x05  B=0x03  ->  respuesta=0x08 (esperado 0x08)
$finish called at time : 21815 ns : File "/home/nicolas/Documents/Facultad/arqui/TP-Arquitectura-comp/ALU.srcs/sim_1/new/tb_top_level.v" Line 67
```

**Figura 11:** Salida de consola de `tb_top_level`.

## 12. Análisis de tiempo: por qué alcanza con buffer + flag

Con un buffer de una sola palabra, un byte se pierde si llega el siguiente antes de que la FSM haya leído el anterior (*overrun*). Una FIFO solo hace falta si eso puede pasar. Para saberlo hay que comparar cuánto tarda la FSM en leer un byte con cada cuánto llega uno nuevo, y verificar que transmitir el resultado no impida recibir la operación siguiente.

Para verlo en simulación se extendió el `tb_top_level` para que mande dos operaciones seguidas: `05 03 20` $(5 + 3)$ y `07 05 20` $(7 + 5)$, con 2 tiempos de bit de pausa entre bytes y a 2 Mbaudios.

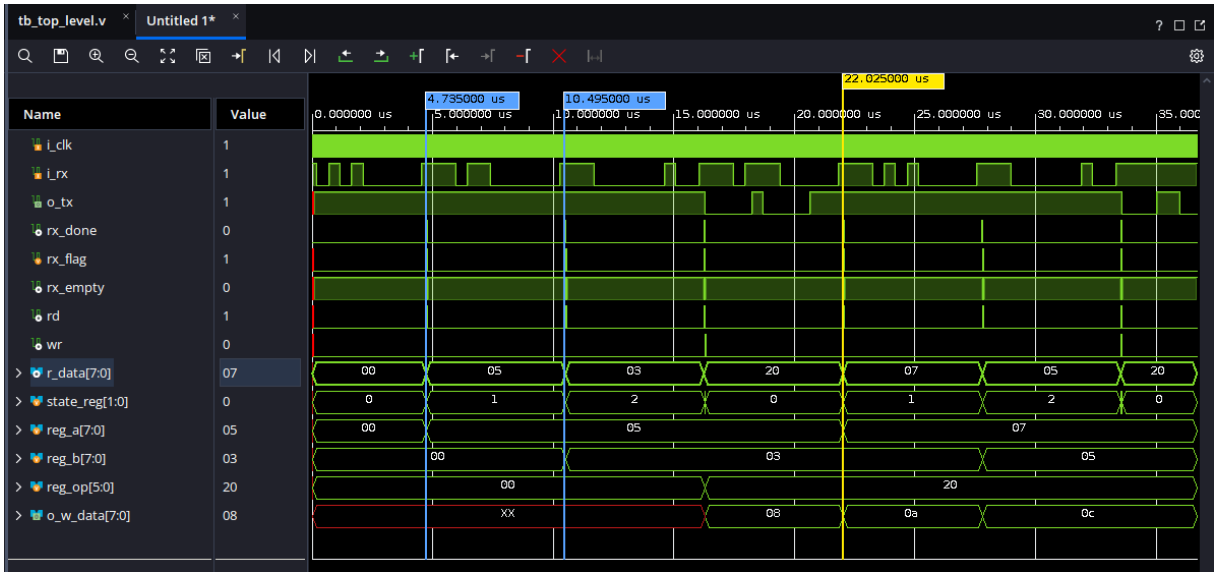



**Figura 12:** Dos operaciones seguidas. Los marcadores azules separan los dos primeros `rx_done` (5760 ns = 576 ciclos). El amarillo marca el `rx_done` del nuevo operando A.

- **Cada byte va a su registro.** `reg_a`, `reg_b` y `reg_op` toman `05`, `03` y `20`, y en la segunda operación `reg_a = 07` y `reg_b = 05`. Ningún byte se carga dos veces ni se saltea.
- **El buffer está casi siempre vacío.** `rx_flag` y `rd` aparecen como rayitas: el byte queda en `rx_buf` un solo ciclo, mientras que entre dos `rx_done` pasan cientos de ciclos (576 en la simulación y 52 000 a 19 200 baudios).

- **Se recibe mientras se transmite.** La trama del nuevo A entra por `i_rx` mientras `o_tx` todavía está enviando `0x08`. Cuando llega su `rx_done` (marcador amarillo, 22,025 µs), la FSM está en `RX_A` desde el final de la operación anterior y lo carga como A, sin confundirlo con B.
- **Solo se escribe el resultado final.** `o_w_data` pasa por `08`, `0A` y `0C`, porque la ALU es combinacional y va mostrando `reg_a + reg_b` a medida que se cargan los registros. Por `o_tx` solo salen `0x08` y `0x0C`, porque `wr` se activa únicamente en `TX_SEND`.

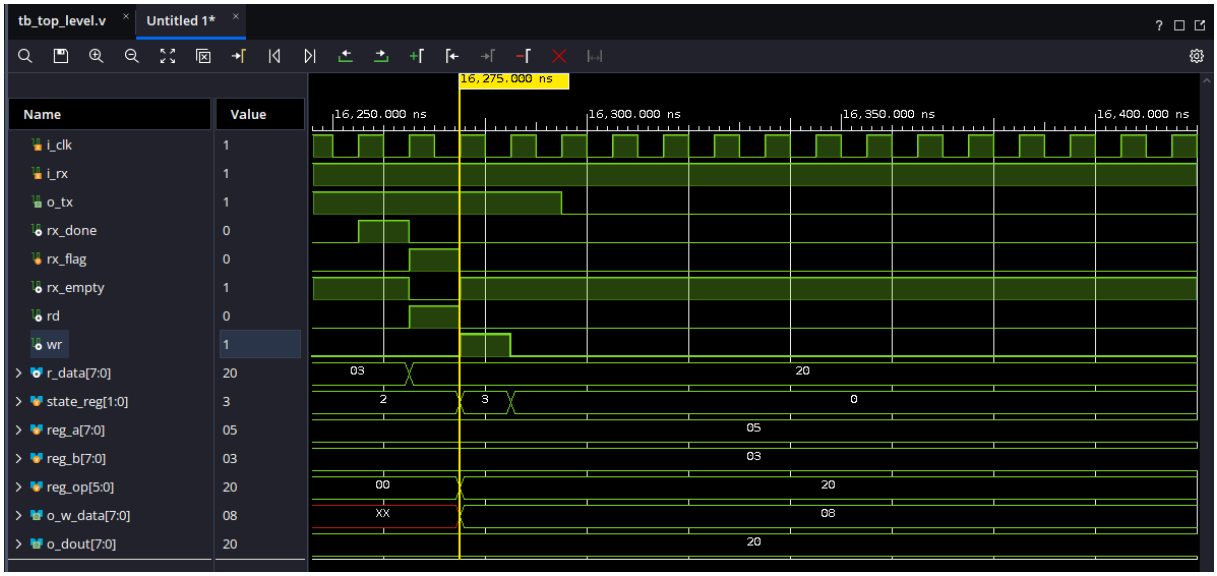



**Figura 13:** Zoom sobre la llegada del opcode de la primera operación.

| Flanco | Qué pasa |
|---|---|
| 16 255 ns | `rx_done`: termina de llegar el opcode |
| 16 265 ns | `r_data = 20`, `rx_flag = 1`, y en el mismo ciclo `rd = 1` (combinacional) |
| 16 275 ns | `reg_op = 20`, `rx_flag` baja, `state_reg` pasa a `TX_SEND` (3), `o_w_data` pasa de `XX` a `08` y se da `wr` |
| 16 285 ns | El resultado queda en `tx_buf` y la FSM **ya vuelve a `RX_A`** (0) |
| 16 295 ns | El `uart_tx` arranca: `o_tx` baja con el bit de start |

**Tabla 8:** Secuencia ciclo por ciclo de la Figura 13.

Este zoom muestra los dos puntos clave:

1. **El buffer RX está lleno un solo ciclo** (16 265 → 16 275 ns). `rd` es combinacional, así que la bandera baja en el mismo flanco en que la FSM cambia de estado y el byte no se lee dos veces.
2. **La FSM no espera a que termine la transmisión.** Entrega el resultado al buffer de TX y un ciclo después ya está en `RX_A`. Los 10 bits de la respuesta los envía el `uart_tx` solo, con su propia copia del byte. Como el receptor y el transmisor son módulos independientes, con líneas separadas, transmitir no frena la recepción.

| | Tiempo (19 200 baudios) |
|---|---|
| Duración de un bit | 52 µs = 5200 ciclos |
| Separación mínima entre dos `rx_done` | 10 bits = 520 µs = 52 000 ciclos |
| Tiempo que el byte queda en `rx_buf` | 1 ciclo = 10 ns |
| Duración de la respuesta | 10 bits = 520 µs |
| Separación mínima entre dos escrituras en `tx_buf` | 30 bits = 1,56 ms |

**Tabla 9:** Tiempos característicos a 19 200 baudios.

Una FIFO sería necesaria si el bloque que consume los datos pudiera estar ocupado más de una trama, por ejemplo un procesador atendiendo otras tareas, o si cada pedido generara más bytes de respuesta que los que recibe.

## 13. Conclusiones

Se implementó una UART completa (receptor, transmisor y generador de baudios) y se la usó para controlar la ALU del trabajo anterior desde una PC, con un circuito de interfaz de buffer + flags entre la UART y la ALU. Se validó cada módulo por separado y luego el sistema completo a través de la línea serie.

- El sobremuestreo x16 con muestreo en el centro de cada bit permite recibir datos de un transmisor con un reloj desconocido, tolerando diferencias de velocidad de varios puntos porcentuales.
- Usar el tick como habilitación mantiene todo el diseño en un único dominio de reloj.
- Separar las FSM en dos bloques `always` con valores por defecto hace el código más claro y evita latches.
- Los buffers con bandera convierten los pulsos de un ciclo de la UART en niveles. Así el bloque que consume los datos no tiene que estar sincronizado con el instante exacto en que llega o sale cada byte, y puede recibir la siguiente operación mientras se transmite el resultado.
- La parametrización permitió reutilizar la ALU sin cambios y simular el sistema completo a 2 Mbaudios modificando un solo parámetro.